

Lab 10&11 - Virtual Private Cloud (VPC) in AWS

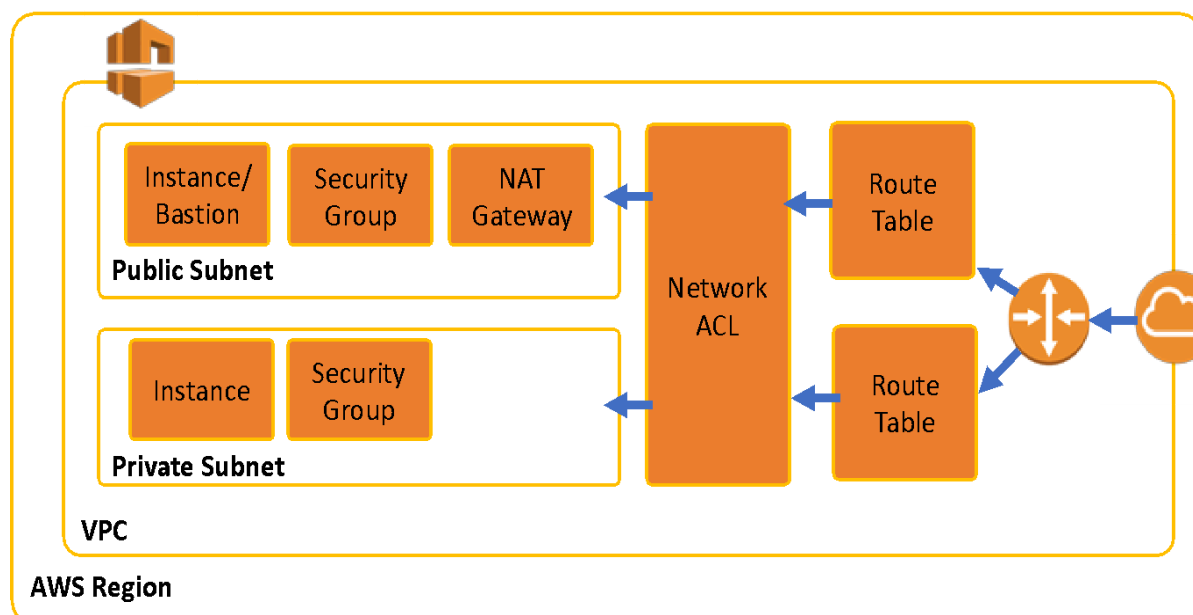
Overview

In the previous labs, we provided hands-on experience with Serverless architecture in AWS and demonstrated how the business logic of a typical web application or even a complete API can be created using AWS Serverless technologies. You learned the basics of working with AWS Lambda, including creating, testing, and deploying Lambda functions through the AWS console. We also explored AWS API Gateway which provides a robust solution for implementing REST and WebSocket services and learned how to create and query a managed NoSQL database service using AWS DynamoDB. In this lab, we will discuss AWS Virtual Private Cloud (VPC) that enables you to build a virtual network in the AWS cloud - no VPNs, hardware, or physical data centres are required. You can define your own network space, and control how your network and the Amazon resources inside your network are exposed to the Internet.

VPC: network level isolation from other things.

Goal

The purpose of this lab is to provide a hands-on experience with VPC. You will learn the basics of creating your own VPC inside AWS. You will also learn how you can easily customise the network configuration of your VPC. You will create a public-facing subnet for your frontend resources (e.g., a web server or load balancer) and place your backend systems (e.g., databases or application servers) in a private-facing subnet with no Internet access. Further, you will learn how you can leverage multiple layers of security including security groups and network access control lists (NACLs) to control access to your Amazon EC2 instances in each subnet. At the end of this lab, we will have the setup shown in the below figure.



Resources

You need an active AWS or AWS Academy account to create resources using the AWS console.

1. **AWS Region:** The entire setup is within an AWS region, which consists of multiple isolated locations known as Availability Zones.
2. **VPC (Virtual Private Cloud):** This is your isolated network within the AWS cloud, where you can launch AWS resources.
3. **Public Subnet:** This subnet is exposed to the internet and typically contains resources that need to be accessible from the internet.
 - **Instance/Bastion:** An EC2 instance that can be accessed from the internet, often used as a gateway to access other instances in the private subnet.
 - **Security Group:** Acts as a virtual firewall to control inbound and outbound traffic for instances.
4. **Private Subnet:** This subnet is not directly accessible from the internet and typically contains resources that should not be exposed to the internet.
 - **Instance:** An EC2 instance that does not have direct access to the internet.
 - **Security Group:** Similar to the public subnet, it controls inbound and outbound traffic for instances in the private subnet.
 - **NAT Gateway:** Allows instances in the private subnet to initiate outbound traffic to the internet, but prevents the internet from initiating connections with those instances.
5. **Network ACL (Access Control List):** Provides an additional layer of security for the VPC by controlling traffic to and from one or more subnets. It serves as a stateless firewall.
6. **Route Table:** Contains a set of rules, called routes, that are used to determine where network traffic is directed. There are route tables associated with both public and private subnets to manage traffic flow.
7. **Internet Gateway:** This component connects the VPC to the internet and allows communication between instances in the VPC and the internet.

Terminology

AWS VPC: Amazon Virtual Private Cloud (Amazon VPC) enables you to launch AWS resources into a virtual network that you have defined. This virtual network closely resembles a traditional network that you would operate in your own data centre, with the benefits of using the scalable infrastructure of AWS.

Network Access Control List (NACL): A network access control list (NACL) is a layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets. You might set up network ACLs with rules similar to your security groups in order to add an additional layer of security to your VPC. Contrary to security groups that are stateful, NACLs are stateless, that is they allow you to do “allow rules” as well as “deny rules”.

Security group is statefull and NACL is stateless. The difference is whether the network will remember the connection or not. Security group will remember the incoming traffic's details, NACL will not remember anything and is a pure rule based traffic filtering.

NAT Gateway: Network Address Translation (NAT) gateway enables your instances in a private subnet to connect to the internet or other AWS services, but prevent the internet from initiating a connection with those instances.

Internet Gateway:

An Internet Gateway (IGW) is a highly available VPC component that allows communication between instances in (VPC) and the internet. It provides a target in your VPC route tables for internet-routable traffic and performs network address translation (NAT) for instances that have been assigned public IP addresses.

Bastion host (Jump box): It is an EC2 instance in a public-facing subnet that will be used to connect to an EC2 instance in a private network.

Task 1: Building your own VPC

Before building your VPC, you need to be familiar with Private IP addresses. Private IP addresses are used within local networks only (behind routers and firewalls) and are not routed on the Internet, and no traffic can be sent to them from the Internet. The following table shows IP range addresses for private IP addresses in class A,B,C allowed in AWS.

IP address range	# of addresses	CIDR block (subnet mask)	Host ID size	Mask bits
10.0.0.0 – 10.255.255.255	16777216	10.0.0.0/8 (255.0.0.0)*	24 bits	8 bits
172.16.0.0 – 172.31.255.255	1048576	172.16.0.0/12 (255.240.0.0)	20 bits	12 bits
192.168.0.0 – 192.168.255.255	65536	192.168.0.0/16 (255.255.0.0)	16 bits	16 bits

*AWS only allows 10.0.0.0/16

CIDR Block

CIDR (Classless Inter-Domain Routing) notation is used to specify IP address ranges. It consists of an IP address, followed by a slash ('/') and a number. The number indicates how many bits are used for the network portion of the address. For example: 10.0.0.0/8: The first 8 bits are the network part, and the remaining 24 bits are available for host addresses.

Subnet Mask

The subnet mask is another way to specify the network portion of an IP address. It works by masking the IP address, with '1' bits indicating the network part and '0' bits indicating the host part. For instance: 255.0.0.0: The first 8 bits (255) are the network part, and the remaining bits are for hosts.

Host ID Size

This indicates the number of bits used to identify hosts within a network. A larger host ID size means more possible host addresses within that subnet.

Mask Bits

This represents the number of bits used for the network portion of the address. Fewer mask bits mean a larger network size.

Application in AWS

When building a VPC in AWS, you can choose a private IP address range based on the needs of your network:

Class A (10.0.0.0/8): Suitable for very large networks with a large number of devices. You have a vast number of possible IP addresses, making it ideal for large enterprise networks.

Class B (172.16.0.0/12): Suitable for medium-sized networks. It provides a good balance between the number of available addresses and network segmentation.

Class C (192.168.0.0/16): Suitable for smaller networks or home networks. It offers a sufficient number of addresses for small to medium-sized environments.

Be careful with what CIDR range to select, as in some collaboration or integration with other applications, need to consider how the network could connect with each other.

VPC cannot be modified after it is created, so if changes need to be made in the future, you will need to create a new one and migrate all current services to it. Need to design and think about how many host addresses you need and how to segment the IP addresses.

You can use this [website](#) to convert CIDR to IPv4.

To build a VPC, go to the Amazon services list and **select VPC under the “Networking & Content Delivery”** section. We are **not** going to use Launch VPC Wizard. From the left bar you browse different relevant resources such as “Your VPCs” which shows 1 default VPC, list of available subnets and their IP ranges, Route tables, Internet Gateways, etc.

Go back to “**Your VPCs**” section and create your own VPC as follows:

Name tag: VPC-demo

IPv4 CIDR block: 10.0.0.0/16

IPAM: IP access management or IP asset management is a framework used to plan, manage, and track IP address allocations in a network. AWS offers a managed IPAM service as part of its VPC (Virtual Private Cloud) offerings, which helps you efficiently manage your IP addresses in AWS and hybrid environments. Sometimes it gets difficult to manage the IP addresses in a large and complex network so IPAM can help to manage it.

Amazon provided IPv6 CIDR block

Tenancy: default

"tenancy" refers to the type of underlying hardware environment in which your EC2 instances run. AWS offers different tenancy options to cater to various use cases, performance needs, and compliance requirements.

Default: resources run on physical servers that may be shared with other AWS customers.

Dedicated: Instances run on hardware that is dedicated to a single customer. Multiple instances from the same account may share the same hardware, but no other AWS accounts will share it.

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

VPC-demo

IPv4 CIDR block [Info](#)
10.0.0.0/16

IPv6 CIDR block [Info](#)
☐ No IPv6 CIDR block
☒ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Network border group
A network border group is a unique group of Zones from where IPv4 and IPv6 IP addresses are advertised. All Availability Zones in this VPC will use this network border group.

us-east-1

Tenancy [Info](#)
Default

This creates a multi-tenant VPC (shared hardware). You can also choose a dedicated option which is an expensive option but gives you dedicated hardware.

When you create a VPC, it does not create any subnet. However, by default it creates a route table, a Network ACL, and a Security Group.

To create a subnet, go to the Subnets tab and create your subnet with **10.0.1.0/24** and name it with block and availability zone as follows:

Please make sure you select the VPC you created (i.e., VPC-demo).

VPC

VPC ID
Create subnets in this VPC.

vpc-05ed4079791c83fc7 (VPC-demo)

Associated VPC CIDRs

IPv4 CIDRs	IPv6 CIDRs
10.0.0.0/16	2600:1f18:615d:4100::/56 (us-east-1)

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

10.0.1.0 - us-east-1a

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

US East (N. Virginia) / us-east-1a

IPv4 CIDR block [Info](#)
10.0.1.0/24

IPv6 CIDR block [Info](#)
Specify whether to assign an IPv6 CIDR block to the subnet.

Do not assign IPv6

Now, create another subnet in a similar way for **10.0.2.0/24** and a new availability zone (e.g., us-east-1b). One of these subnets will be our public one and the other one is private and not accessible from outside. Check this [link](#) to see how many IPs should be available for different subnet masking places.

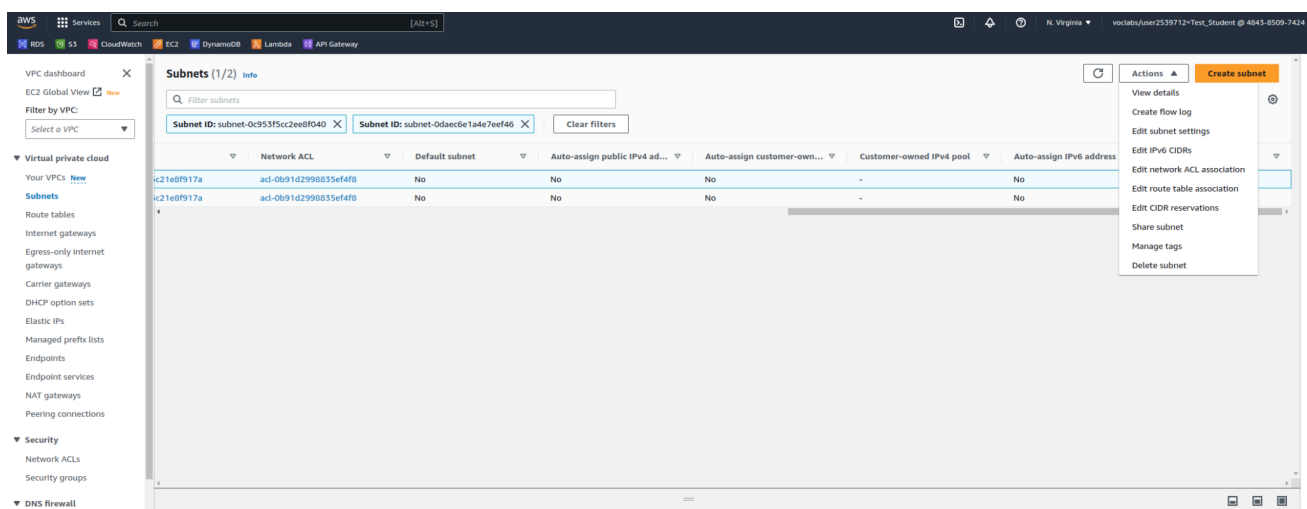
NOTE: We cannot create a subnet that is spanning across multiple availability zones.

Question: Why is the total number of addresses in each subnet not 254 (Check available IPv4 column)?

AWS will reserve some of the IP addresses for monitoring (cloudwatch). Will reserve 3 IP addresses.

Find the answer here: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html#VPC_Sizing

Both subnets have auto IP disabled (scroll your window to the right to see the column with this information). **Select your first subnet (10.0.1.0) and from the “Actions” list select “Edit Subnet Settings” and enable auto-sign IPv4.**



Now we need an **Internet Gateway** to communicate with our VPC. From the left bar, select **Internet Gateways** and create a **new one** named **“MyIGW”**. By default, it is in detached mode. From the **“Actions”** list select **“Attach to VPC”** and select your VPC and attach.

NOTE: We cannot attach two Internet Gateways to our VPCs. You can test it.

In AWS, a VPC can only have one Internet Gateway attached to it at any given time. This restriction exists for several reasons related to design, functionality, and the principles of routing and network management.

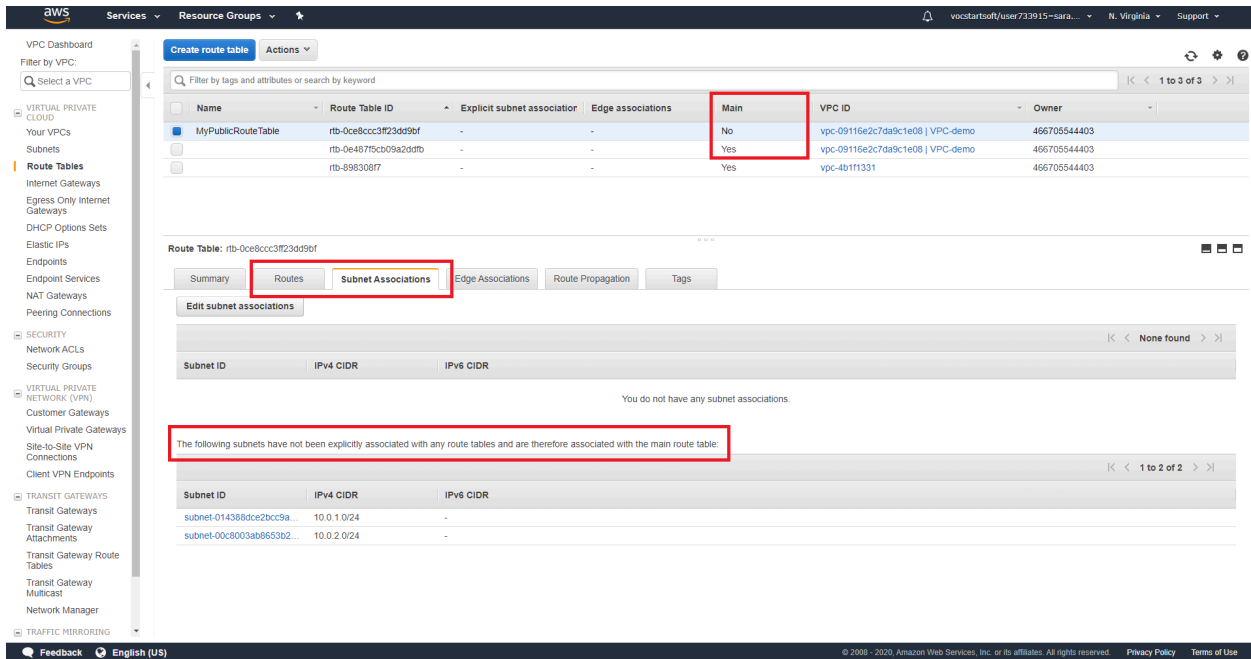
Now that we have our resources, we need to connect them together. **Go to the “Route Table” tab.** Click on the route which is attached to your VPC. **If you click on that and go to routes tabs. You can see there are two routes here. These routes are allowing your subnets to communicate with each other.**

If you go to the subnet associations tab, you should see a message like:

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table.

This means that all subnets that we have by default would be associated with the main route table. **We need to keep our private subnet private and create a route to the Internet for our public subnets.**

Go to the “Route Table” tab and create a new route table with the name of “MyPublicRouteTable” for your VPC. You can see the default table is main and newly created subnets are associated with this one. **But for the newly created table we need to specifically associate subnets.**



The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Subnet ID	IPv4 CIDR	IPv6 CIDR
subnet-014388dce2bcc59a...	10.0.1.0/24	-
subnet-00c8003ab6653b2...	10.0.2.0/24	-

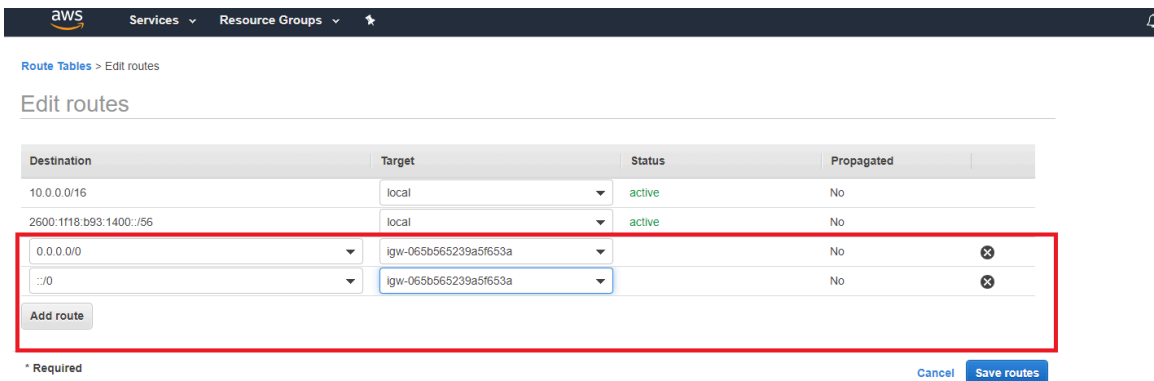
Select “MyPublicRouteTable” route table and edit routes from “Routes” tab and add the following:

0.0.0.0/0 with the Target that will be our Internet Gateway.

::/0 with the Target that will be our Internet Gateway.

One is for ipv4 and the other is for ipv6

We added two new routes (the second one for IPv6) and the target is selected as “Internet Gateway”.



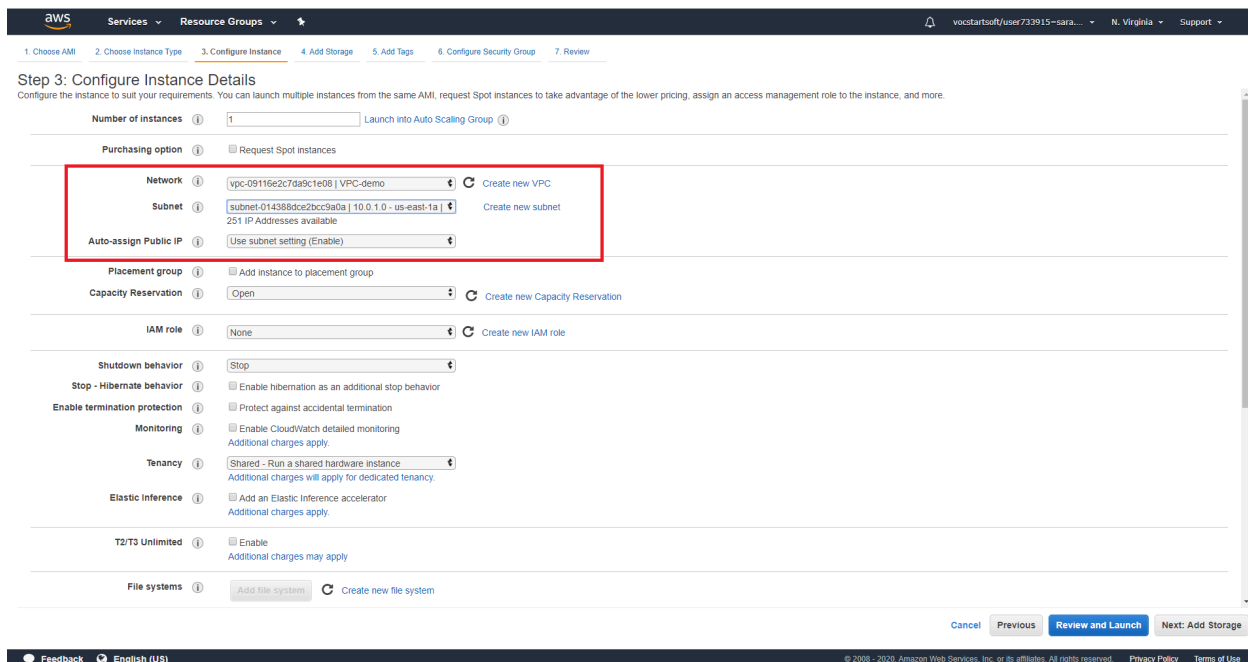
Destination	Target	Status	Propagated
10.0.0.0/16	local	active	No
2600:1f18:b93:1400::/56	local	active	No
0.0.0.0/0	igw-065b565239a5f653a		No
::/0	igw-065b565239a5f653a		No

Any subnet that is associated with this table will automatically be public. **Edit subnet association and select 10.0.1.0 subnet as your public subnet.** Now we have one subnet associated with our public subnet and another one to the main subnet. To test our environment, we create two EC2 instances, one for each subnet.

Task 2: Building EC2 Instances in VPC subnets

Go to the EC2 dashboard and launch an **Ubuntu** instance (**Ubuntu Server 22.04 LTS(HVM)**, **SSD Volume Type**, **free-tier eligible**) with default values as the previous lab except for the following parts:

Network settings updated to our VPC network with **public subnet**:



Step 3: Configure Instance Details

Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of the lower pricing, assign an access management role to the instance, and more.

Number of instances: 1 Launch into Auto Scaling Group

Purchasing option: ☐ Request Spot instances

Network: vpc-09116e2c7da9c1e08 | VPC-demo Create new VPC

Subnet: subnet-014388dce2bcc9a0a | 10.0.1.0 - us-east-1a Create new subnet

Auto-assign Public IP: Use subnet setting (Enable)

Placement group: ☐ Add instance to placement group

Capacity Reservation: Open Create new Capacity Reservation

IAM role: None Create new IAM role

Shutdown behavior: Stop

Stop - Hibernate behavior: ☐ Enable hibernation as an additional stop behavior

Enable termination protection: ☐ Protect against accidental termination

Monitoring: ☐ Enable CloudWatch detailed monitoring Additional charges apply.

Tenancy: Shared - Run a shared hardware instance Additional charges will apply for dedicated tenancy

Elastic inference: ☐ Add an Elastic Inference accelerator Additional charges apply.

T2/T3 Unlimited: ☐ Enable Additional charges may apply

File systems: Add file system Create new file system

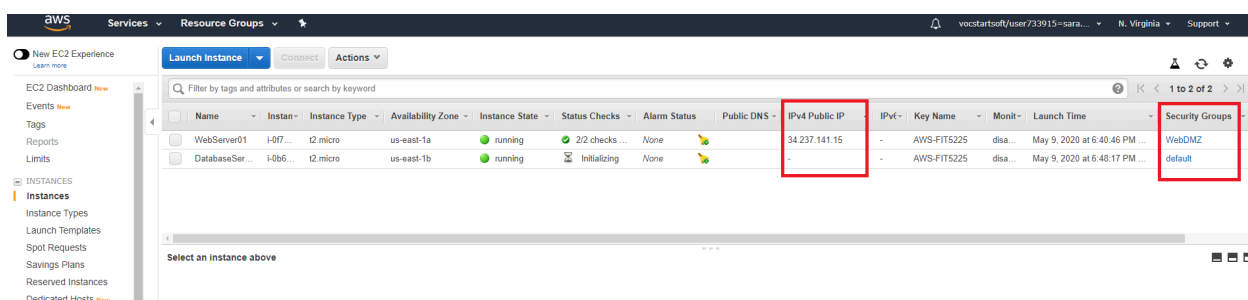
Cancel Previous Review and Launch Next: Add Storage

Feedback English (US) © 2008 - 2020 Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy Policy Terms of Use

Add a tag with **Name:WebServer01**. Also, create a new security group with the name “**WebDMZ**” and add a rule for HTTP from “anywhere” to that. Launch your instance after you complete these steps.

NOTE: Security Groups do not span across VPCs.

Create the second instance with similar configs except tag with **Name:DatabaseServer**, for network settings select your **private subnet** (Make sure **Auto-assign Public IP is disabled for this one**) and select an existing security group and use default security group instead of WebDMZ.



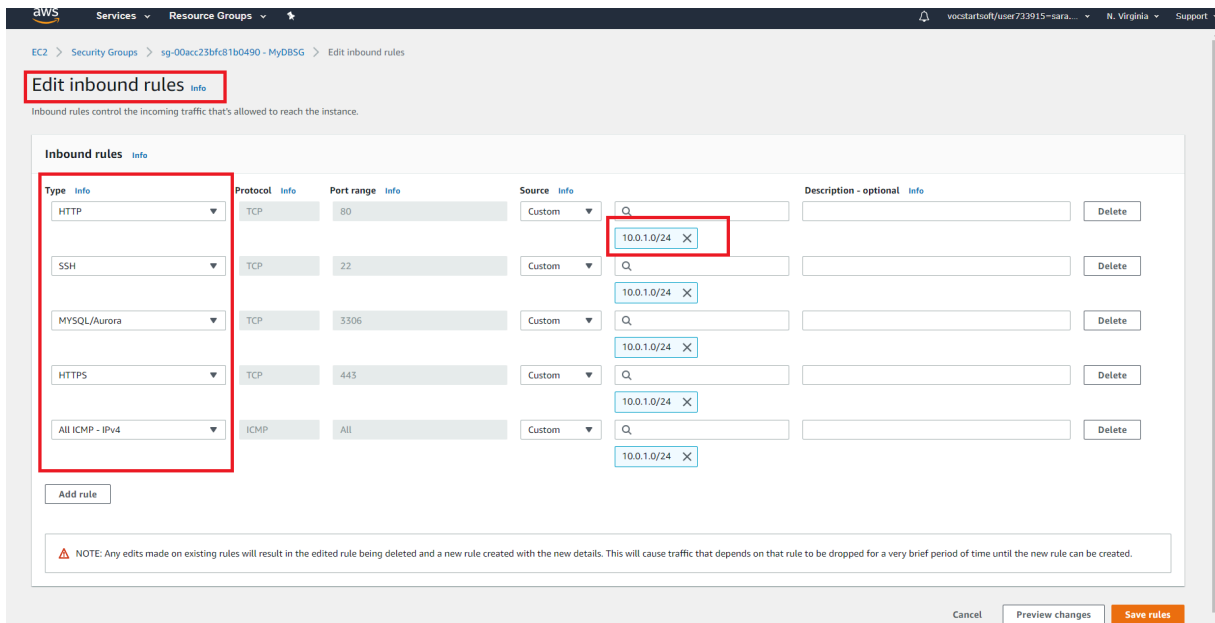
Name	Instan...	Instance Type	Availability Zone	Instance State	Status Checks	Alarm Status	Public DNS	IPv4 Public IP	IPv6	Key Name	Monit...	Launch Time	Security Groups
WebServer01	i-07...	t2.micro	us-east-1a	running	2/2 checks ...	None		34.237.141.15	-	AWS-FIT5225	disa...	May 9, 2020 at 6:40:46 PM ...	WebDMZ
DatabaseSer...	i-0b6...	t2.micro	us-east-1b	running	Initializing	None		-	-	AWS-FIT5225	disa...	May 9, 2020 at 6:48:17 PM ...	default

Now SSH to your web server with the assigned public IP address to check your connection. You can see that your EC2 instance in the private network does not have a public address, therefore you can't access it via SSH directly over the Internet.

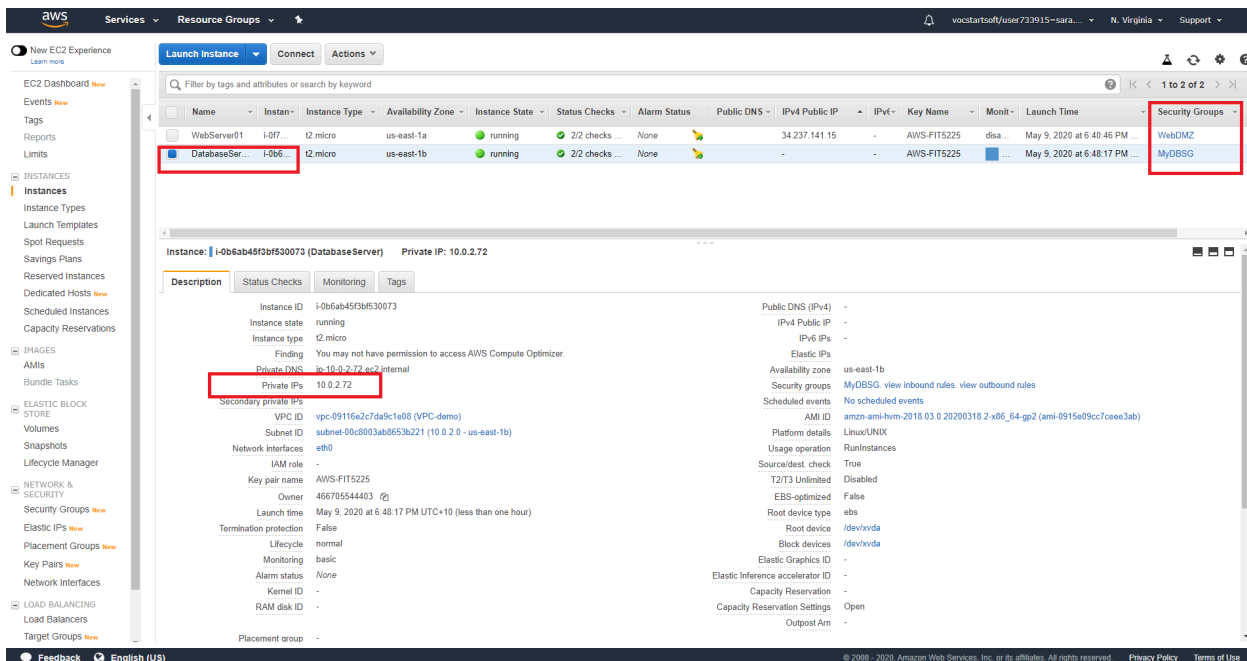
At this point, we have created our VPC and we are able to connect to our public subnet. However, we cannot yet access our private subnet. To make our public and private subnet connected we need to perform a few more steps.

First, create a new security group (from the left bar of your dashboard under “Network & Security”) named “MyDBSG” for your VPC (select your VPC from the drop-down) and with following **Inbound** rules which allow different connection protocols from your public subnet:

Open HTTP, HTTPS, SSH, MYSQL and All ICMP- IPv4 for the source 10.0.1.0/24.



Now move your database server to the new security group. Select your DatabaseServer instance and from “Actions” > “Security” select “Change Security Groups” and remove the default one and add “MyDBSG” instead and save it.



Copy the private IP of your database server and try to ping it from your Web server terminal.

Make sure you have already ssh-ed to your public EC2 instance. From there do a ping to your EC2 instance in the private instance. (Replace X with the correct number)

```
$ ping 10.0.2.X
```

Now you should be able to see that ping is working.

Bastion Host

A bastion host is a special-purpose computer on a network designed and configured to withstand attacks. The computer generally hosts a single application and all other services are removed to reduce the threat to that computer. It is hardened through ACL and security groups.

To connect to your private instance from your PC through a Bastion Host:

Option 1) `ssh -i {your_private_key} -A -J ubuntu@{jump_host_public_ip} ubuntu@{destination_host_private_ip}`

- **-A:** Enables forwarding of the authentication agent connection. This is useful if you want to use your local SSH keys on the remote host.
- **-J ubuntu@{jump_host_public_ip}:** Specifies the jump host (bastion host).
- **SSH connects to the jump host using the provided private key and user credentials.**
- **From the jump host, SSH then connects to the destination host using the same private key and user credentials.**

If it doesn't work for some OS, use a proxy command instead:

Option 2) `ssh -o ProxyCommand="ssh -i {private_key} -W %h:%p ubuntu@{jump_host_public_ip}" -i {private_key} ubuntu@{destination_private_ip}`

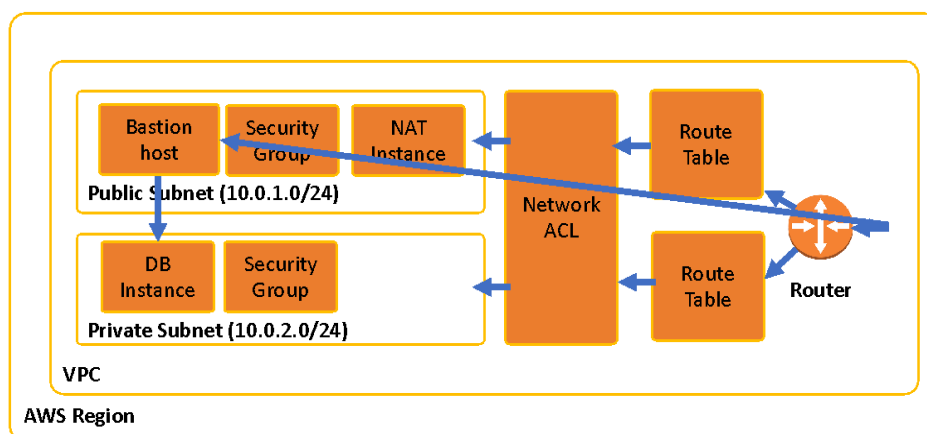
- **-o ProxyCommand="ssh -i {private_key} -W %h:%p ubuntu@{jump_host_public_ip}":** Specifies a command to use as a proxy for the connection.
 - **-o ProxyCommand:** Option to specify a proxy command.
 - **ssh -i {private_key} -W %h:%p ubuntu@{jump_host_public_ip}:** The command that will run to establish the proxy connection.
 - **ssh -i {private_key}:** Uses the specified private key for authentication.
 - **-W %h:%p:** Specifies that SSH should forward the connection from the jump host to the destination host (%h is a placeholder for the destination host and %p for the destination port).
 - **ubuntu@{jump_host_public_ip}:** Connects to the jump host using the provided credentials and private key.

- **-i {private_key}**: Specifies the private key file to use for authentication on the final leg of the connection.
- **ubuntu@{destination_private_ip}**: Specifies the destination host to connect to.
 - Replace **ubuntu** with the username for the destination host.
 - Replace **{destination_private_ip}** with the private IP address of the destination host.

How It Works:

1. SSH connects to the jump host using the provided private key and user credentials.
2. The jump host then forwards the connection to the destination host, using the private key again for authentication.

Your tutor will explain those commands. Option 2 allows you to use two private keys (one for bastion, one for the target instance) to further enhance security.



Question: What do you think would happen if you do an apt-get update on your private EC2 instance?

Unreachable because private instance not connected to internet yet

Task 3: Building a NAT Instance

To enable your EC2 instances in our private subnet to be able to get Internet access without making it public, we can use: **NAT Gateways** and **NAT instances**. NAT Gateway is a more scalable and reliable way of doing that. However, it charges an hourly rate and per GB of data processed (link: [NAT Gateway Pricing](#)). For NAT instances, you need to pay for the EC2 instance fee only (\$0.045/hr NAT Gateway vs \$0.011/hr t2.micro, no traffic processing charge for NAT instance).

Thus, we will do it through a NAT Instance. Setting and configuration of your VPC might be quite similar in both cases.

1. Open the Amazon EC2 console.
2. Create an instance. **Go to community AMIs** and search for "NAT". Select the latest (**amzn-ami-vpc-nat**) by **publish date**. Choose instance type and go ahead to the configure Instance page.
3. Make sure you **select your VPC and your public subnet (10.0.1.0)** and "Auto-assign Public IP" is enabled.
4. Next you can add a tag to your instance, e.g., Key: **Name** and value: **Nat_Instance**.

5. Select **“default”** and **“WebDMZ”** security groups
6. Set your key and finally launch your instance.

You can find more info here: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_NAT_Instance.html

Each EC2 instance performs source/destination checks by default. This means that the instance must be the source or destination of any traffic it sends or receives. However, a NAT instance must be able to send and receive traffic when the source or destination is not itself. Therefore, you must disable source/destination checks on the NAT instance.

To disable source/destination checking using the console, select your NAT instance, choose **Actions>Networking>Change Source/Dest** and select **“Stop”**.

To make sure our DB instance can talk with our NAT instance, we need to create a route in **our default route table**:

1. Open the Amazon VPC console.
2. Open **Route Tables**.
3. Select your main route table for your custom VPC (the one with Yes in the main column).
4. Go to the Routes tab and **Edit Routes** and **Add Route**.
 - a. Add 0.0.0.0/0 as the **Destination** and Instance as **Target** and select your NAT_Instance.

Now we have a route out to the Internet in the main route table. Let's check if our EC2 Instance in the private network has access to the Internet.

SSH into your DB instance in the private subnet through the bastion host and try to run apt-get update.

Now you can see that your DB instance has access to the Internet.

The problem with NAT instance is that it is a single EC2 instance and it can easily get overwhelmed if many resources are using this instance as their gateway. As mentioned earlier, the better way is to use an AWS NAT Gateway.

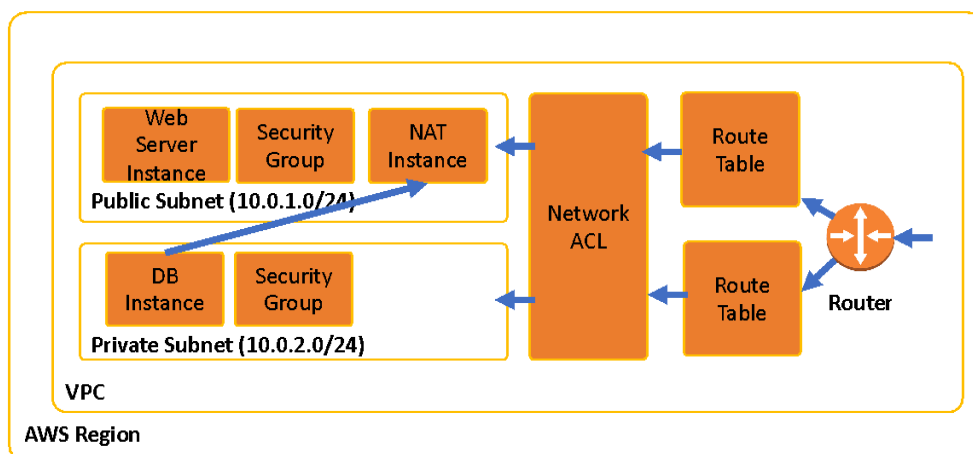
Now you can go and stop your NAT_Instance and check your Internet connection on your DB instance. You should see that it does not work.

Make sure you start it again for the next task.

NOTE: You cannot use a NAT Gateway as a Bastion host.

Task 4: NACL vs Security Group

In this task, we are going to compare the Access Control List with Security Groups. So far, we have the following architecture.



Network ACL and security groups operate on different layers.

- scope of control: security groups serve as a virtual firewall for instances to control inbound and outbound traffic at the instance level. NACLs associated with subnets within a VPC so provides a layer of security at subnet level.
- state: security group rules are stateful - meaning that if inbound traffic is allowed to an instance, the response traffic is automatically allowed, regardless of outbound rules. NACLs rules are stateless - which requires both inbound and outbound rules to be explicitly set.

1. Open the Amazon VPC console.
2. Open **Network ACLs** tab under **Security**.
3. Select the ACL associated with your custom VPC.
4. Look at **Inbound Rules** and **Outbound Rules**.

Question: Each network ACL rule has a rule number, what is the role of this number?

Determine the order in which the rules are evaluated when network traffic attempts to enter or exit a subnet. Network ACL rules are evaluated in ascending numerical order based on their rule numbers. This order dictates how traffic is processed and whether it is allowed or denied. Rule Prioritization: Administrators can manipulate the order of rules by assigning specific numbers.

Question: What does a default rule whose rule number is an asterisk mean?

The rule number indicated by an asterisk (*) is a catch-all rule that is applied to any network traffic that does not match any of the other specified rules in the Network ACL. This rule acts as the last line of defense or default setting for managing traffic. It ensures that if none of the other user-defined rules in the ACL match the traffic, this default rule will apply. Typically Set to Deny.

1. Create a new ACL using the Create **network ACL** button.
 - a. For **Name tag** give MyWebNACL.

- b. Select your custom VPC for **VPC**
- c. Create your new ACL.
- d. Select your MyWebNACL and check **Inbound** and **Outbound** rules.

Let's check how ACL works. SSH to your web server instance and install apache2:

```
sudo -i
```

then:

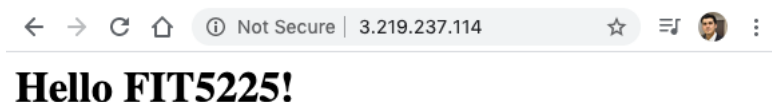
```
apt update && sudo apt install apache2 -y
```

```
cd /var/www/html/
```

```
mv index.html index2.html
```

```
echo '<html><h1>Hello FIT5225!</h1></html>' > index.html
```

2. Now copy the IP address of your Web server instance to your browser address bar. You should get the following page.



Currently, our subnets are associated with the default ACL that allows all traffic in. **Now we would like to associate it with MyWebNACL ACL**

1. Open the Amazon VPC console, Then **Network ACLs**.
2. Select MyWebNACL.
3. Then select **Subnet associations** and then **Edit subnet associations**.
4. Select your public subnet (10.0.1.0).

Now, try to access your web server by refreshing the page in your browser. **You will see it does not connect as your ACL blocks all traffic. Now we are going to edit our MyWebNACL.**

1. Open **Inbound rules** and **Edit inbound rules** add the following rules and save.

Edit inbound rules

Network ACL acl-0c250866ae19f3f14

Rule #	Type	Protocol	Port Range ⓘ	Source ⓘ	Allow / Deny	
100	Custom TCP Rule ▾	TCP (6) ▾	80	0.0.0.0/0	ALLOW ▾	✕
200	Custom TCP Rule ▾	TCP (6) ▾	443	0.0.0.0/0	ALLOW ▾	✕
300	Custom TCP Rule ▾	TCP (6) ▾	22	0.0.0.0/0	ALLOW ▾	✕

Add Rule

* Required

Cancel Save

2. Open **Outbound rules** and **Edit Outbound rules** add the following rules and save.

Edit outbound rules

Network ACL: acl-0c250866ae19f3f14

Rule #	Type	Protocol	Port Range	Destination	Allow / Deny	
100	HTTP (80)	TCP (6)	80	0.0.0.0/0	ALLOW	✕
200	HTTPS (443)	TCP (6)	443	0.0.0.0/0	ALLOW	✕
300	Custom TCP Rule	TCP (6)	1024 - 65535	0.0.0.0/0	ALLOW	✕

Add Rule

* Required

Cancel Save

Ephemeral ports, ranging from 1024 to 65535, are temporary ports assigned automatically on the client side during direct communication with a server's well-known port. These ports can also be used on the server side to maintain ongoing communications with a client after an initial connection to a well-known service port.

Ports 1024-65535 are ephemeral ports. Ephemeral ports are allocated automatically for the client end as an end client's port assignment in direct communication with a well-known port used by a server. On servers, ephemeral ports may also be used as the port assignment on the server end of a communication. This is done to continue communications with a client that initially connected to one of the server's well-known service listening ports ([Wikipedia](https://en.wikipedia.org/wiki/Ephemeral_port)). The client that initiates the request chooses the ephemeral port range. The range varies depending on the client's operating system. A NAT gateway uses ports 1024-65535. For more info on ephemeral ports check AWS documentation: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html#nacl-ephemeral-ports>

Now if you refresh your web page you should be able to see your webpage again.

Please note that as a packet comes to the subnet, it will be evaluated against the inbound rules of the ACL that the subnet is associated with (starting at the top of the list of rules and moving to the bottom). This means that rules with lower numbers in the list have more priority than rules with higher numbers.

To test this, add a deny rule to your ACL (for inbound rule) and deny access for port 80 and your IP address. Type what is my ip in Google. Then, add a deny rule as follows:

400 Custom TCP Rule TCP (6) 0 Your IP HERE/32 ALLOW ✕

If you refresh your page you still see that the webpage is loading. This is because you have rule 100 that allows 80 from any source.

NOTE: The Network ACL is performed in chronological order. That is, a network ACL contains a numbered list of rules that are evaluated in order, starting with the lowest numbered rule, to determine whether traffic is allowed in or out of any subnet associated with the network ACL.

If we change our deny rule to 99, we will see that the webpage is not loading.

/Users/jcdu3/Desktop/Teaching/FIT5225/a1/inputfolder